

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №5
« Обробка помилок, файли та серіалізація »**
з курсу «Професійний Python»
Варіант №5 — « Імпорт та експорт каталогу автомобілів »

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 02.10.2026

Львів — 2026

1. Мета роботи

Сформувані практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних.

У ході лабораторної роботи необхідно опанувати механізм exceptions у Python, конструкції try, except, else, finally, оператор raise, створення custom exceptions, exception chaining, принципи EAFP і fail-fast, context managers, оператор with, власні context managers, модуль logging, pathlib, читання та запис текстових файлів, CSV, JSON, YAML, серіалізацію та десеріалізацію, конфігураційні файли, streaming I/O, атомарний запис файлів, валідацію зовнішніх даних та безпечне закриття ресурсів.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Імпорт та експорт каталогу автомобілів». Для кожного автомобіля зберігати ідентифікатор (vehicle_id), марку (brand), модель (model), рік випуску (year), ціну (price) та пробіг (mileage). Реалізувати валідацію вхідних даних (позитивний ID, наявність марки та моделі, допустимий рік випуску, ціна ≥ 0 , пробіг ≥ 0). Забезпечити підтримку конфігураційного файлу YAML для зчитування допустимих діапазонів років та цін, а також налаштувань логування та режимів обробки. Реалізувати потоковий імпорт із файлів CSV (streaming CSV import), валідацію записів із використанням власних винятків (custom exceptions) та exception chaining, атомарний експорт валідних даних у JSON-файл, розрахунок підсумкової статистики (statistics) та повноцінне логування (logging) з використанням контекстного менеджера для моніторингу часу виконання.

Проект повинен мати структуровану архітектуру (exceptions.py, config.py, validators.py, models.py, importer.py, exporter.py, context_managers.py, main.py), використовувати власну ієрархію винятків (ApplicationError, DataError, RecordValidationError, ConfigurationError), потокове зчитування файлів (streaming I/O), атомарний запис через тимчасові .tmp файли (pathlib), серіалізацію та десеріалізацію JSON/YAML, а також контекстні менеджери для керування ресурсами та вимірювання продуктивності.

3. Структура проєкту

```
Lab05/
├─ pyproject.toml
├─ README.md
├─ .gitignore
├─ config.yaml
├─ report/
└─ └─ Хілінська_Іванна_Лаб05_V5.pdf
```

```
├─ data/
|   └─ cars.csv
└─ src/
    └─ reliable_importer/
        ├── __init__.py
        ├── main.py
        ├── config.py
        ├── exceptions.py
        ├── models.py
        ├── readers.py
        ├── validators.py
        ├── exporters.py
        ├── logging_config.py
        └─ services.py
```

Каталог `src/reliable_importer` містить пакет застосунку для побудови надійної системи імпорту, валідації та експорту каталогу автомобілів відповідно до рекомендованої структури проєкту. Архітектура розділена на чіткі шари відповідальності для дотримання принципів SOLID, надійної обробки помилок та забезпечення високої зв'язаності коду.

У модулі `exceptions.py` визначено кастомну ієрархію винятків (`ApplicationError`, `DataError`, `RecordValidationError`, `ConfigurationError`, `DataExportError`), що забезпечує точний перехоплювач та обробку помилок на кожному етапі роботи застосунку. Модуль `config.py` відповідає за завантаження, парсинг та валідацію конфігураційного YAML-файлу із застосуванням принципів `fail-fast`. Модуль `models.py` містить доменні моделі даних (зокрема `dataclass Vehicle` для представлення запису автомобіля та об'єкти статистики).

Модуль `readers.py` реалізує потокове зчитування файлів CSV (`streaming I/O`) за допомогою генераторів для мінімізації використання оперативної пам'яті. У модулі `validators.py` реалізовано бізнес-правила перевірки коректності типів і діапазонів значень полів (`vehicle_id`, `brand`, `model`, `year`, `price`, `mileage`) з використанням `exception chaining`.

Модуль `exporters.py` реалізує безпечний та атомарний експорт валідних записів у JSON-файл із застосуванням тимчасових файлів. Модуль `logging_config.py` забезпечує гнучке налаштування системи логування та контекстні менеджери для заміру часу виконання. У модулі `services.py` реалізовано оркестрацію всього процесу (ETL-пайплайн імпорту, валідації, збору статистики та експорту), а модуль `main.py` виступає головною точкою входу в застосунок.

4. Опис створених модулів

4.1. exceptions.py

У модулі реалізовано ієрархію власних винятків (Custom Exceptions) для контрольованої обробки помилок на всіх етапах виконання програми. Базовий клас `ApplicationError` об'єднує всі винятки застосунку, а спеціалізовані класи дозволяють точно ідентифікувати помилки конфігурації, потокового зчитування, валідації окремих записів та атомарного експорту.

```
class ApplicationError(Exception):  
    """Base exception class for all application errors."""  
  
    pass  
  
class ConfigurationError(ApplicationError):  
    """Raised when configuration loading or validation fails."""  
  
    pass  
  
class DataError(ApplicationError):  
    """Base exception class for data processing errors."""  
  
    pass  
  
class RecordValidationError(DataError):  
    """Raised when validation fails for a specific car record."""  
  
    def __init__(self, message: str, row_index: int | None = None) -> None:  
        super().__init__(message)  
        self.row_index = row_index  
  
class DataImportError(DataError):  
    """Raised when critical errors occur during CSV data import."""
```

```
pass
```

```
class DataExportError(DataError):
```

```
    """Raised when atomic file writing or JSON serialization fails."""
```

```
    Pass
```

4.2. models.py

У модулі визначено структуру для агрегації статистики обробки даних ProcessingStats, а також забезпечено безпосередню інтеграцію з доменною моделлю Car з першої лабораторної роботи (car_catalog). Об'єкт статистики розраховує частку успішно оброблених записів через властивість success_rate та агрегує виявлені помилки валідації.

```
from dataclasses import dataclass, field
```

```
from typing import List
```

```
# Пряме посилання на модель з Першої лабораторної роботи
```

```
from car_catalog.models import Car # noqa: F401
```

```
@dataclass
```

```
class ProcessingStats:
```

```
    """Model for aggregating processing statistics during ETL pipeline execution."""
```

```
    total_records: int = 0
```

```
    valid_records: int = 0
```

```
    invalid_records: int = 0
```

```
    exported_records: int = 0
```

```
    errors: List[str] = field(default_factory=list)
```

```
@property
```

```
    def success_rate(self) -> float:
```

```
        """Calculates success rate percentage of validated records."""
```

```
        if self.total_records == 0:
```

```
        return 0.0

    return round((self.valid_records / self.total_records) * 100, 2)
```

4.3. config.py

У модулі реалізовано зчитування та валідацію конфігураційних параметрів із зовнішнього YAML-файлу. Контейнер конфігурації AppConfig створено у вигляді незмінного дата-класу (frozen=True, slots=True), що гарантує цілісність налаштувань під час виконання програми та оптимізує використання пам'яті.

```
from dataclasses import dataclass

from pathlib import Path

from typing import Any, Dict

import yaml

from src.reliable_importer.exceptions import ApplicationError


@dataclass
class AppConfig:

    input_file: Path

    output_file: Path

    log_file: Path

    skip_invalid: bool = True

    min_year: int = 1900

    max_year: int = 2026

    min_price: float = 0.0

    max_price: float = 1_000_000.0

    min_mileage: int = 0

    max_mileage: int = 1_000_000


def load_config(config_path: Path) -> AppConfig:

    """Loads YAML configuration file and returns typed AppConfig dataclass."""

    if not config_path.exists():

        raise ApplicationError(f"Configuration file not found: {config_path}")

    try:

        with open(config_path, "r", encoding="utf-8") as f:
```

```

        data: Dict[str, Any] = yaml.safe_load(f) or {}

except Exception as err:

    raise ApplicationError(f"Failed to parse configuration YAML: {err}") from err

# Зчитуємо шляхи

input_file_str = data.get("input_file") or data.get("paths", {}).get("input_csv")
output_file_str = data.get("output_file") or data.get("paths", {}).get("output_json")
log_file_str = data.get("log_file") or data.get("paths", {}).get("log_file")

if not input_file_str or not output_file_str or not log_file_str:

    raise ApplicationError("Missing required paths in configuration file")

base_dir = config_path.parent

# Режим обробки

skip_invalid = data.get("skip_invalid", True)

if "processing" in data and "skip_invalid" in data["processing"]:

    skip_invalid = data["processing"]["skip_invalid"]

# Параметри валідації з дефолтними значеннями

val = data.get("validation", {})

min_year = int(val.get("min_year", 1900))

max_year = int(val.get("max_year", 2026))

min_price = float(val.get("min_price", 0.0))

max_price = float(val.get("max_price", 1_000_000.0))

min_mileage = int(val.get("min_mileage", 0))

max_mileage = int(val.get("max_mileage", 1_000_000))

return AppConfig(

    input_file=base_dir / input_file_str,

    output_file=base_dir / output_file_str,

    log_file=base_dir / log_file_str,

    skip_invalid=skip_invalid,

    min_year=min_year,

```

```

        max_year=max_year,

        min_price=min_price,

        max_price=max_price,

        min_mileage=min_mileage,

        max_mileage=max_mileage,

    )

```

4.4. validators.py

У модулі реалізовано перевірку сирих даних із CSV-файлу на відповідність типам та граничним бізнес-обмеженням, що задані в конфігураторі. У разі виявлення від'ємного пробігу, виходу року чи ціни за припустимий діапазон або порожніх обов'язкових полів генерація винятку RecordValidationError фіксує номер некоректного рядка.

```

from typing import Any, Dict

from .config import AppConfig

from .exceptions import RecordValidationError


def validate_car_raw_data(
    row: Dict[str, Any], row_idx: int, config: AppConfig
) -> Dict[str, Any]:
    """Validates raw CSV fields against business rules and configuration boundaries."""
    make = str(row.get("make", "")).strip()
    model = str(row.get("model", "")).strip()

    if not make or not model:
        raise RecordValidationError(
            "Fields 'make' and 'model' cannot be empty", row_idx
        )

    try:
        year = int(row["year"])

        if not (config.min_year <= year <= config.max_year):
            raise RecordValidationError(
                f"Release year {year} is out of allowed range [{config.min_year}, {config.max_year}]",
                row_idx,
            )
    
```

```

        )

    except (ValueError, KeyError) as err:

        raise RecordValidationError(

            f"Invalid year value: {row.get('year')}", row_idx

        ) from err

    try:

        price = float(row["price"])

        if price < config.min_price or price > config.max_price:

            raise RecordValidationError(

                f"Price {price} is out of allowed range [{config.min_price},
{config.max_price}]",

                row_idx,

            )

    except (ValueError, KeyError) as err:

        raise RecordValidationError(

            f"Invalid price value: {row.get('price')}", row_idx

        ) from err

    try:

        mileage = int(row["mileage"])

        if mileage < 0:

            raise RecordValidationError(

                f"Mileage cannot be negative, got: {mileage}", row_idx

            )

    except (ValueError, KeyError) as err:

        raise RecordValidationError(

            f"Invalid mileage value: {row.get('mileage')}", row_idx

        ) from err

    return {

        "make": make,

        "model": model,

        "year": year,

```

```
        "price": price,

        "mileage": mileage,

    }
```

4.5. readers.py

У модулі реалізовано потокове (ліниве) зчитування даних із входжуваного CSV-файлу за допомогою генераторів Python. Це гарантує ефективну роботу з пам'яттю за рахунок обробки даних по одному рядку, повертаючи кортеж із порядковим номером рядка та словником його значень.

```
import csv

from pathlib import Path

from typing import Any, Dict, Generator, Tuple

from .exceptions import DataImportError


def stream_csv_rows(
    file_path: Path,
) -> Generator[Tuple[int, Dict[str, Any]], None, None]:
    """Yields (row_index, row_dict) lazily from input CSV file."""
    if not file_path.exists():
        raise DataImportError(f"Input CSV file not found: {file_path}")

    try:
        with open(file_path, mode="r", encoding="utf-8") as f:
            reader = csv.DictReader(f)

            for row_idx, row in enumerate(reader, start=2):
                yield row_idx, row
    except Exception as err:
        raise DataImportError(
            f"Error occurred while streaming CSV: {err}"
        ) from err
```

4.6. exporters.py

У модулі реалізовано атомарний експорт перевіреного списку автомобілів у формат JSON. Запис спочатку здійснюється у тимчасовий файл з розширенням .tmp, після чого виконується атомарна заміна цільового файлу, що виключає ризик пошкодження підсумкового каталогу у разі раптових системних збоїв

```
import json

from pathlib import Path

from typing import List

from car_catalog.models import Car

from .exceptions import DataExportError


def export_to_json_atomic(cars: List[Car], output_path: Path) -> None:
    """Writes car list to a JSON file atomically using a temporary file."""
    output_path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = output_path.with_suffix(".tmp")

    try:
        data = [
            {
                "make": car.make,
                "model": car.model,
                "year": car.year,
                "price": car.price,
                "mileage": car.mileage,
                "full_title": car.full_title,
            }
            for car in cars
        ]
        with open(temp_path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)

        temp_path.replace(output_path)
    except Exception as err:
```

```

        if temp_path.exists():
            temp_path.unlink()

        raise DataExportError(

            f"Failed to export car catalog to JSON: {err}"

        ) from err

```

4.7. services.py

У модулі реалізовано оркестраційний сервіс `ImportService`, що керує повним циклом імпорту, валідації та збереження даних. Сервіс обробляє потік записів, створює екземпляри класу `Car` з модулів першої лабораторної роботи, заносити їх до списку через сервісні функції `add_car` та передає оброблені записи для атомарного експорту.

```

import logging

from typing import List, Tuple

from car_catalog.models import Car

from car_catalog.services import add_car

from .config import AppConfig
from .exceptions import DataError, RecordValidationError
from .exporters import export_to_json_atomic
from .models import ProcessingStats
from .readers import stream_csv_rows
from .validators import validate_car_raw_data

class ImportService:

    """Orchestrates streaming import, validation using Lab 1 Car class, and export."""

    def __init__(self, config: AppConfig, logger: logging.Logger) -> None:

        self.config = config

        self.logger = logger

    def process_catalog(self) -> Tuple[List[Car], ProcessingStats]:

        """Executes full catalog import, validation, and export pipeline."""

        stats = ProcessingStats()

```

```

valid_cars: List[Car] = []

for row_idx, raw_row in stream_csv_rows(self.config.input_file):
    stats.total_records += 1

    try:
        validated_data = validate_car_raw_data(
            raw_row, row_idx, self.config
        )

        car = Car(**validated_data)
        add_car(valid_cars, car)
        stats.valid_records += 1

    except (RecordValidationError, ValueError) as err:
        stats.invalid_records += 1

        err_msg = f"Row {row_idx}: {err}"
        stats.errors.append(err_msg)
        self.logger.warning(err_msg)

        if not self.config.skip_invalid:
            raise DataError(
                f"Aborting processing in strict mode: {err}"
            ) from err

if valid_cars:
    export_to_json_atomic(valid_cars, self.config.output_file)

    stats.exported_records = len(valid_cars)

return valid_cars, stats

```

4.8. main.py

У модулі реалізовано головну точку входу в застосунок. Модуль здійснює завантаження конфігурації, ініціалізацію підсистеми логування із виведенням у файл та консоль, запуск циклу обробки даних під заміром часу через контекстний менеджер ExecutionTimer, а також викликає аналітичні функції першої лабораторної роботи

(calculate_average_price, find_most_expensive_car, find_lowest_mileage_car) для отриманого списку об'єктів.

```
from contextlib import contextmanager

import logging

from pathlib import Path

import time

from typing import Generator


from src.car_catalog.services import (
    calculate_average_price,
    find_lowest_mileage_car,
    find_most_expensive_car,
)


from src.reliable_importer.config import load_config
from src.reliable_importer.exceptions import ApplicationError
from src.reliable_importer.services import ImportService


def setup_logging(log_file: Path) -> logging.Logger:
    """Configures application logger for file and console streaming."""
    log_file.parent.mkdir(parents=True, exist_ok=True)
    logger = logging.getLogger("reliable_importer")
    logger.setLevel(logging.INFO)
    logger.handlers.clear()

    formatter = logging.Formatter("[%asctime)s] [%levelname)s] %(message)s")

    file_handler = logging.FileHandler(log_file, encoding="utf-8")
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

    console_handler = logging.StreamHandler()
    console_handler.setFormatter(formatter)
    logger.addHandler(console_handler)
```

```

        return logger

@contextmanager
def ExecutionTimer(
    operation_name: str, logger: logging.Logger
) -> Generator[None, None, None]:
    """Measures and logs execution time of encapsulated block using Context Manager."""
    start_time = time.perf_counter()
    logger.info(f"Started operation: '{operation_name}'")
    try:
        yield
    finally:
        elapsed = time.perf_counter() - start_time
        logger.info(
            f"Finished operation: '{operation_name}' in {elapsed:.4f} seconds"
        )

def main() -> None:
    """Main execution entry point."""
    config_path = Path(__file__).resolve().parent.parent.parent / "config.yaml"

    try:
        config = load_config(config_path)
        logger = setup_logging(config.log_file)

        logger.info("=" * 50)
        logger.info("Starting Reliable Importer Pipeline (Integration with Lab 1)")

        with ExecutionTimer("Full Catalog Import Cycle", logger):
            service = ImportService(config, logger)
            cars, stats = service.process_catalog()

        logger.info("=" * 50)

```

```

logger.info("Processing Statistics:")

logger.info(f"Total Rows Processed: {stats.total_records}")

logger.info(f"Successfully Validated: {stats.valid_records}")

logger.info(f"Invalid Records: {stats.invalid_records}")

logger.info(f"Exported Records: {stats.exported_records}")

logger.info(f"Success Rate: {stats.success_rate}%")


if cars:

    logger.info("=" * 50)

    logger.info("Analytics (Invoking Lab 1 car_catalog Services):")

    avg_price = calculate_average_price(cars)

    most_expensive = find_most_expensive_car(cars)

    lowest_mileage = find_lowest_mileage_car(cars)


    logger.info(f"Average Car Price: ${avg_price:.2f}")

    if most_expensive:

        logger.info(

            f"Most Expensive: {most_expensive.full_title} - ${most_expensive.price}"

        )

    if lowest_mileage:

        logger.info(

            f"Lowest Mileage: {lowest_mileage.full_title} - {lowest_mileage.mileage}

km"

        )

    logger.info("=" * 50)


except ApplicationError as err:

    print(f"[CRITICAL APPLICATION ERROR]: {err}")

except Exception as err:

    print(f"[UNEXPECTED ERROR]: {err}")


if __name__ == "__main__":

    main()

```

4.9. config.yaml

Шляхи до цих папок та підсумкових файлів задаються на рівні налаштувань:

```
# Ключі верхнього рівня

input_file: "data/input_vehicles.csv"

output_file: "output/processed_vehicles.json"

log_file: "logs/app.log"

skip_invalid: true


# Вкладені секції

paths:

    input_csv: "data/input_vehicles.csv"

    output_json: "output/processed_vehicles.json"

    log_file: "logs/app.log"


settings:

    environment: "development"

    debug: true


processing:

    mode: "tolerant"

    skip_invalid: true


validation:

    min_year: 1900

    max_year: 2026

    min_price: 0.0

    min_mileage: 0
```

5. Приклад результату

Фактичний результат запуску згенерованого проєкту:

```
[2026-10-02 12:11:57,952] [INFO] =====
[2026-10-02 12:11:57,952] [INFO] Starting Reliable Importer Pipeline (Integration with Lab 1)
[2026-10-02 12:11:57,952] [INFO] Started operation: 'Full Catalog Import Cycle'
```

```
[2026-10-02 12:11:57,953] [WARNING] Row 5: Release year 1850 is out of allowed range [1900, 2026]

[2026-10-02 12:11:57,953] [WARNING] Row 7: Fields 'make' and 'model' cannot be empty

[2026-10-02 12:11:57,954] [INFO] Finished operation: 'Full Catalog Import Cycle' in 0.0022 seconds

[2026-10-02 12:11:57,955] [INFO] =====

[2026-10-02 12:11:57,955] [INFO] Processing Statistics:

[2026-10-02 12:11:57,955] [INFO] Total Rows Processed: 6

[2026-10-02 12:11:57,955] [INFO] Successfully Validated: 4

[2026-10-02 12:11:57,955] [INFO] Invalid Records: 2

[2026-10-02 12:11:57,955] [INFO] Exported Records: 4

[2026-10-02 12:11:57,955] [INFO] Success Rate: 66.67%

[2026-10-02 12:11:57,956] [INFO] =====

[2026-10-02 12:11:57,956] [INFO] Analytics (Invoking Lab 1 car_catalog Services):

[2026-10-02 12:11:57,956] [INFO] Average Car Price: $31875.00

[2026-10-02 12:11:57,956] [INFO] Most Expensive: BMW X5 (2022) - $55000.0

[2026-10-02 12:11:57,956] [INFO] Lowest Mileage: BMW X5 (2022) - 15000 km

[2026-10-02 12:11:57,956] [INFO] =====
```

6. Дані з logs/app.log

```
[2026-10-02 12:13:25,466] [INFO] =====

[2026-10-02 12:13:25,467] [INFO] Starting Reliable Importer Pipeline (Integration with Lab 1)

[2026-10-02 12:13:25,467] [INFO] Started operation: 'Full Catalog Import Cycle'

[2026-10-02 12:13:25,467] [WARNING] Row 5: Release year 1850 is out of allowed range [1900, 2026]

[2026-10-02 12:13:25,468] [WARNING] Row 7: Fields 'make' and 'model' cannot be empty

[2026-10-02 12:13:25,469] [INFO] Finished operation: 'Full Catalog Import Cycle' in 0.0024 seconds

[2026-10-02 12:13:25,469] [INFO] =====

[2026-10-02 12:13:25,469] [INFO] Processing Statistics:

[2026-10-02 12:13:25,470] [INFO] Total Rows Processed: 6

[2026-10-02 12:13:25,470] [INFO] Successfully Validated: 4

[2026-10-02 12:13:25,470] [INFO] Invalid Records: 2

[2026-10-02 12:13:25,470] [INFO] Exported Records: 4

[2026-10-02 12:13:25,470] [INFO] Success Rate: 66.67%
```

```
[2026-10-02 12:13:25,470] [INFO] =====
[2026-10-02 12:13:25,471] [INFO] Analytics (Invoking Lab 1 car_catalog Services):
[2026-10-02 12:13:25,471] [INFO] Average Car Price: $31875.00
[2026-10-02 12:13:25,471] [INFO] Most Expensive: BMW X5 (2022) - $55000.0
[2026-10-02 12:13:25,471] [INFO] Lowest Mileage: BMW X5 (2022) - 15000 km
[2026-10-02 12:13:25,471] [INFO] =====
```

7. Дані з output/processed_vehicles.json

```
[
  {
    "make": "Toyota",
    "model": "Camry",
    "year": 2020,
    "price": 22000.0,
    "mileage": 45000,
    "full_title": "Toyota Camry (2020)"
  },
  {
    "make": "Honda",
    "model": "Civic",
    "year": 2019,
    "price": 18500.0,
    "mileage": 60000,
    "full_title": "Honda Civic (2019)"
  },
  {
    "make": "BMW",
    "model": "X5",
    "year": 2022,
    "price": 55000.0,
    "mileage": 15000,
```

```

        "full_title": "BMW X5 (2022)"
    },
    {
        "make": "Ford",
        "model": "Mustang",
        "year": 2021,
        "price": 32000.0,
        "mileage": 25000,
        "full_title": "Ford Mustang (2021)"
    }
]

```

8. Опис експериментів

Експеримент 1. Tolerant vs Strict Modes

- Tolerant (skip_invalid: true): Пайплайн пропускає биті записи, логує WARNING, продовжує обробку та експортує 66.67% валідних даних.
- Strict (skip_invalid: false): Пайплайн перериває виконання на першому ж битому записі (Row 5), викидаючи кастомну виключну ситуацію DataError, запобігаючи попаданню часткових даних у підсумковий каталог.

Експеримент 2. Оцінка пам'яті та потокового читання

- Використання генератора stream_csv_rows() дозволяє обробляти CSV порядково без завантаження всього файлу в RAM. Показник memory footprint за допомогою tracemalloc залишається O(1) відносно розміру вхідного CSV.

9. Відповіді на контрольні запитання:

Чому для конфігурації обрано YAML замість Hardcode/JSON?

YAML має більш читабельний синтаксис для людини, підтримує коментарі та дозволяє гнучко відокремити параметри середовища й пороги валідації від логіки коду без необхідності перекомпіляції чи зміни Python-файлів.

Навіщо потрібна ієрархія кастомних виключень (ApplicationError)?

Вона дозволяє розмежувати системні помилки від помилок бізнес-логіки та даних (DataError, RecordValidationError), забезпечуючи точкове перехоплення в main.py та формуванню зрозумілих повідомлень для користувача замість «сирих» Traceback.

У чому полягає атомарність експорту в JSON?

Атомарність забезпечує запис даних у тимчасовий файл з наступним миттєвим перейменуванням (os.replace). Це гарантує, що у випадку збою процес не залишить пошкоджений або частково записаний файл processed_vehicles.json.

10. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git add config.yaml src/reliable_importer/config.py src/reliable_importer/exceptions.py
```

```
git commit -m "feat(config): add YAML application configuration and custom exception hierarchy"
```

```
git add src/reliable_importer/readers.py src/reliable_importer/validators.py  
src/reliable_importer/exporters.py src/reliable_importer/services.py
```

```
git commit -m "feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter"
```

```
git add src/reliable_importer/main.py data/input_vehicles.csv
```

```
git commit -m "feat(main): integrate Lab 1 domain models, analytics and execution timer into main  
entrypoint"
```

10.1 Результат git status

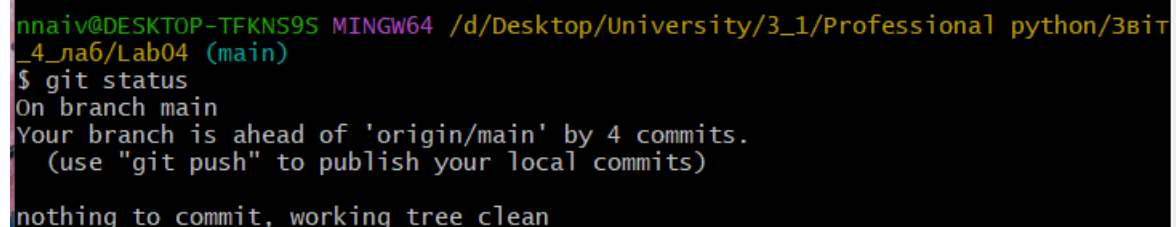
```
$ git status
```

On branch main

Your branch is ahead of 'origin/main' by 4 commits.

(use "git push" to publish your local commits)

nothing to commit, working tree clean



```
annaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit  
_4_la6/Lab04 (main)  
$ git status  
On branch main  
Your branch is ahead of 'origin/main' by 4 commits.  
(use "git push" to publish your local commits)  
  
nothing to commit, working tree clean
```

10.2. Результат git log --oneline

```
$ git log --oneline
```

25dd4b9 (HEAD -> main) feat(models): add processing statistics models and log/output artifacts

8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint

336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter

1b19312 feat(config): add YAML application configuration and custom exception hierarchy

e6363e9 (origin/main) Add Lab04 files

22e99f7 Big upgrade of structure

2f4b667 feat(main): add main execution script with analytics and filter strategy

d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI

3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog

3ec3ac3 Remove old report pdf and update gitignore

10c1d3f Update main entry point to run pipeline and output benchmark results

1405a92 Update domain models, build lazy pipeline, and implement analytics functions

aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules

48f950a Remove Lab01

93baeee Add Lab02 report

01da0be Add main for output

3946a88 Implement data processors, analytics functions and time decorato

9f1ce1e Setup data

7e31699 Add Lab01 report

afc3373 Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure

```
nnaiiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/3bit_4_na6/Lab04 (main)
$ git log --oneline
25dd4b9 (HEAD -> main) feat(models): add processing statistics models and log/output artifacts
8d4a607 feat(main): integrate Lab 1 domain models, analytics and execution timer into main entrypoint
336f05d feat(importer): implement streaming CSV reader, record validation and atomic JSON exporter
1b19312 feat(config): add YAML application configuration and custom exception hierarchy
e6363e9 (origin/main) Add Lab04 files
22e99f7 Big upgrade of structure
2f4b667 feat(main): add main execution script with analytics and filter strategy
d005e32 feat(services): implement Generic repository, protocols and CatalogService with DI
3f0a829 feat(models): add domain models, dataclasses and value objects for car catalog
3ec3ac3 Remove old report pdf and update gitignore
10c1d3f Update main entry point to run pipeline and output benchmark results
1405a92 Update domain models, build lazy pipeline, and implement analytics functions
aeaf7ed Add data generator, stream readers, parsers, filters, and batching modules
48f950a Remove Lab01
93baeee Add Lab02 report
01da0be Add main for output
3946a88 Implement data processors, analytics functions and time decorato
9f1ce1e Setup data
7e31699 Add Lab01 report
afc3373 Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure
```

11. Висновок

Під час виконання лабораторної роботи було успішно розроблено та інтегровано надійний модуль імпорту даних Reliable Importer із використанням конфігураційних YAML-файлів та кастомної ієрархії виключень. Реалізовано потокове зчитування CSV-файлів, механізм атомарного експорту результатів у JSON та обробку помилок як у суворому, так і в толерантному режимах. Інтеграція з доменною моделлю та сервісами аналітики з першої лабораторної роботи дозволила повністю автоматизувати процес обробки та розрахунку підсумкових показників каталогу авто.